GAMIFY: CODEMONS & PLATFORM MILESTONE

Comprehensive Technical & Feature Milestone Summary for the Engineering Team

Repository: sidd-289/gamify	**Target Branch:** main (Synchronized)	**Milestone Date:** October 2026
Core Stack: Next.js 15, React, TypeScript	**Retro Visuals:** HTML5 Canvas (160x144 GB)	**Execution Engine:** Web Worker Blob Sandbox

1. Executive Summary

This milestone report details the successful delivery of the **Codemons Retro Code Combat Arena** and its integration with the **Gamify** educational platform. The system bridges classic 8-bit Game Boy combat mechanics with real-time, browser-isolated JavaScript programming challenges. Students write, test, and debug algorithmic code inside a sandboxed IDE to command their starter Codemons against rogue bug monsters. All subsystems have been fully implemented, validated across comprehensive test suites, and pushed to origin/main.

2. Dual-Column Arena Architecture

The application features a responsive two-column split-screen viewport optimized for desktop displays while rigorously preserving retro aesthetic constraints (strict Game Boy palette, zero border-radius, hard pixel borders):

- **Left Column (55% Width — #game-panel):** Renders the 160x144 pixel art canvas scene, title mascot preview, starter selection screen, animated battle scene, live HUD status (Points 0/120, Turns Left 8), color-shifting HP bars, and DOM typewriter dialogue.
- **Right Column (45% Width — #code-panel):** Contains the 3-section interactive Code Lab: (1) Question Picker with 3 difficulty cards, (2) Code Editor with line-number gutter and turn timer, and (3) Output Console showing test assertion logs.
- **State Decoupling & Security:** The game simulation runs on the main browser thread while user code compilation and execution occur strictly within a sandboxed Web Worker to guarantee safety and prevent UI freezes.

3. 160x144 Canvas Visual Engine & Starters

- **Authentic 160x144 Pixel Grid:** Rendered on a native low-resolution canvas scaled crisply using CSS image-rendering: pixelated. Features matrix floor grids, arena horizon lines, and retro banner styling.
- **16x16 Pixel Sprite Matrices:** Authored in editable JavaScript palette index grids (using the 14-color Game Boy palette):
  - **Pyrawr:** Python-themed starter (100 HP, 1.0x attack multiplier, balanced archetype).
  - **Jaskull:** JavaScript/TypeScript-themed starter (90 HP, 1.1x attack multiplier, glass cannon archetype).
  - **Cplusaur:** C++-themed starter (120 HP, 0.9x attack multiplier, tank archetype).
  - **Bugmon:** Adversary glitch monster (130 HP, 1.0x attack power, Segfault Slam & NullPointer Crash attacks).
- **Dynamic Color-Shifting HP Bars:** Smooth animation transitions changing color dynamically: Green (>50%), Yellow (25-50%), and Red (<25%) with exact numeric readouts (e.g. 80/100).
- **Combat Dialogue:** Interactive DOM typewriter box (typeText) animating letter-by-letter combat reports.
- **Screen Manager:** Clean transitions across Screen 1 (Title), Screen 2 (Starter Select with keyboard Left/Right/Enter navigation), and Screen 3 (Active Battle).

4. Code Panel Subsystems & Editor Implementation

4.1. Question Picker (#question-picker)

At the start of each player turn, 3 challenge cards are drawn side-by-side representing distinct difficulty tiers: **EASY** (10 Base DMG, Green accent), **MEDIUM** (20 Base DMG, Amber accent), and **HARD** (35 Base DMG, Red accent). Challenges are randomly drawn from the curriculum pool while tracking battleUsedChallenges to guarantee zero repeats within a single battle session. Clicking a card engages a blinking retro border (.picker-card.selected), locks the other two cards (.picker-card.locked), and loads the starter code into the editor.

4.2. Code Editor Area & Synchronized Gutter (#editor-area)

- **Monospace Code Editor:** Styled with the retro VT323 font, green caret, and syntax clarity.
- **Line Number Gutter (#editor-gutter):** Synchronizes in real time with line counts on input and vertical scroll offset on scroll.
- **Tab Key Indentation:** Intercepts the Tab key to inject 2 spaces (' ') and advance the cursor without switching focus away from the editor.
- **Turn Timer Bar:** Enforces strict turn countdowns: **60 seconds** for Easy, **90 seconds** for Medium, and **120 seconds** for Hard. Features warning color thresholds (amber < 50%, flashing red pulse < 25%). If the timer expires, the turn fails and Bugmon retaliates automatically.
- **Action Controls:**
  - ■ RUN TESTS: Submits code to the Web Worker for test suite validation.
  - ? HINT (-5 PTS): Deducts 5 points from the player's HUD score and displays the challenge hint.
  - ■ RESET: Re-populates the editor with the original starter code signature.

4.3. Output Console (#output-console)

Provides structured feedback line-by-line using retro color tags:
- TEST X: PASS (Styled in vibrant green for passed assertions)
- TEST X: FAIL (expected Y, got Z) (Styled in warning red/yellow with expected vs received diffs)
- ERROR: [message] (Syntax and runtime errors highlighted with a red badge)
- TIMEOUT: possible infinite loop (Generated when code execution exceeds the 2-second worker ceiling)

5. Safe Code Runner & Web Worker Sandboxing

Security and UI thread stability were prioritized by implementing a non-blocking sandbox architecture in public/js/runner.js:

- **Blob Web Worker Spawning:** Converts the worker script string into an in-memory Blob and URL (URL.createObjectURL). The main browser thread never evaluates or executes student-written code directly.
- **Recursive Deep Equality:** Validates return values against test criteria using recursive comparisons for primitives, arrays, nested matrices, and objects.
- **2-Second Infinite Loop Guard:** Enforces a hard 2000ms timeout via setTimeout on worker execution. If a student writes an infinite loop (e.g. while(true){}), the worker is terminated immediately (worker.terminate()) and reports TIMEOUT: possible infinite loop without freezing the browser.

6. JavaScript Challenge Curriculum Dataset

A curriculum of 18 JavaScript programming challenges was authored across 6 programming topics in public/data/challenges.json, providing balanced progression from introductory variables to recursive algorithms:

Difficulty	Base DMG	Timer	Topic Scope	Sample Challenges Included
EASY (6)	10 DMG	60s	Variables, Conditionals, Functions	Sum of Two, Even/Odd, String Length, Positive Check, Max of Two, Greeting Generator
MEDIUM (6)	20 DMG	90s	Loops, Arrays, String Manipulation	Array Sum, Count Occurrences, Reverse String, FizzBuzz Variant, Evens Only, Longest Word
HARD (6)	35 DMG	120s	Debugging, Recursion, Algorithms	Fix Negative Max Bug, Recursive Factorial, Fibonacci, Palindrome, Two Sum, Flatten 2D Array

7. Platform Enhancements & Supporting Systems

- **1-Click Quick Demo Accounts:** Integrated demo login buttons on /login for **Student** (student@gamify.dev), **Faculty** (faculty@gamify.dev), and **Admin** (admin@gamify.dev), switchable via NEXT_PUBLIC_DEMO_MODE.
- **Arcade Lounge (play.html) & Student Hub Linkage:** Developed the dedicated Play Section hub with retro scanlines and CRT styling. Added direct launch cards on the student learning dashboard (/student/learn).
- **Automated Streak Freezes:** Automated monthly streak freeze allocations per missed day with streak_freeze audit event logs and strict 7-day bonus payout calculations (streak_7_bonus_<student>_<streak_start_date>).
- **Daily Quiz Expiration & XP Lifecycle:** Implemented midnight IST expiration for unfinished quiz attempts without awarding unearned XP, unblocking subsequent day quiz eligibility with dynamic 'Earn up to 40 XP' badges.
- **SSR Hydration & Turbopack Resilience:** Resolved hydration mismatches for animated components (BottomNav MagneticDock, StaggeredMenu, AcademySidebar) and fixed font preconnect headers.

8. Quality Assurance & Verification Audit

Verification Test Suite	Component	Result	Notes
Challenge Solution Validation	challenges.json	**PASSED (18/18)**	All 18 challenge solutions pass 100% of assertion tests
Safe Runner & Worker Sandbox	runner.js	**PASSED**	Validated pass assertions, fail diffs, and 2s infinite loop cutoff
DOM Elements & Selector Audit	index.html & style.css	**PASSED (22/22)**	All required DOM IDs and 23 CSS classes verified present
Next.js TypeScript Compiler	TypeScript (tsc)	**PASSED (0 Errors)**	npx tsc --noEmit compiled cleanly across all application routes
Git Repository Status	Git Working Tree	**CLEAN & SYNCED**	All code committed and pushed to origin/main

9. Key Delivered Git Commits

Commit Hash	Type / Scope	Description
`416b2c9`	**feat(codemons)**	Build interactive code panel with question picker, editor, turn timer, safe runner, and output console
`07469b4`	**feat(auth)**	Add 1-click quick demo accounts for student, faculty, and admin on login page
`d146fc0`	**feat(play)**	Add Play section arcade lounge page and integrate Codemons card into student learn page
`c6d7730`	**feat(codemons)**	Implement full retro layout, screen manager, starter selection, and style system
`b1de2df`	**feat(codemons)**	Add 160x144 battle scene engine, sprite arrays, and animations
`1c6ebe8`	**feat(codemons)**	Add starter sprite assets, data catalog, and questions pool
`2358092`	**fix(quiz)**	Expire unfinished daily quiz attempts at midnight IST without XP and unblock next day quiz
`f65b3c6`	**fix(streaks)**	Automate monthly freeze allocation per missed day and log individual streak_freeze audit events

10. Recommended Next Milestones

- **Leaderboards & XP Sync:** Persist Codemons battle victories and score tallies into student Supabase XP records.
- **Multiplayer Byte Battles:** WebSocket / Supabase Realtime synchronized duel mode where two learners race to compile bug-free solutions.
- **Expanded Codemons Roster:** Additional mascots representing Rust, Go, and Java with unique signature abilities and movesets.

Prepared By: Antigravity AI Engineering Assistant	**Reviewed By:** Project Lead & Engineering Team
Status: All Features Delivered & Operational	**Target Environment:** Production / Staging Ready